编译系统上机大作业预备工作
LLVM IR 编程与 MLIR 进阶探索

袁煜杰（合作成员：张龙飞，3769lkj）

2026 年 10 月 11 日

摘要

本报告记录本人负责的基本要求二 LLVM IR 编程与进阶 MLIR 探索。独立设计六组 SysY 程序并手写 LLVM IR，使用结构验证、独立预期结果、参考程序、本机执行及 RISC-V 模拟执行检查语义；33 组输入和 mem2reg 对照共 236 项检查全部通过。进阶部分研究 AscendNPU IR 官方文档，另实际执行上游 MLIR 的 Linalg、SCF、CF、LLVM 方言降低以及 LLVM IR 翻译，VecAdd 的 16 个元素全部正确。当前未配置 Ascend NPU 与 CANN，因此专有方言路线仅报告文档研究。本报告是本人部分的完整附件；小组最终共同报告还需合入搭档的分析和手写汇编实验。

目录

1 任务与分工 **2**

2 基本要求二：手写 LLVM IR **2**

2.1 任务范围与实现方法 2
2.2 实验环境与复现流程 2
2.3 示例设计与特性覆盖 3
2.4 短路控制流与可观测副作用 3
2.5 循环、函数与数据表示 3
2.6 mem2reg 观察与结果验证 4
2.7 结论与搭档交接 4

3 进阶要求：MLIR 多级中间表示探索 **5**

3.1 探索目标与证据边界 5
3.2 AscendNPU IR VecAdd 的方言与层级 5
3.3 可执行的上游 VecAdd 设计 5
3.4 逐层 lowering 与检查点 6
3.5 末端验证与分析 6

4 交付物与后续整合 **7**

1 任务与分工

课程原始资料包括大作业说明、预备工作课件、SysY 定义、运行时说明及文法补充。两人合作完成一份预备作业报告，开头说明分工，两人分别提交。袁煜杰负责 LLVM IR 示例、验证脚本、MLIR 进阶探索及相应报告；张龙飞负责基本要求二的手写 RISC-V 汇编，已有工作流程实验由其继续完善。本附件不替代尚待整合的小组共同报告，也不代表已实现完整 SysY 编译器。

2 基本要求二：手写 LLVM IR

2.1 任务范围与实现方法

本部分由袁煜杰完成，对应预备工作的基本要求二中的 LLVM IR 编程任务。在已有循环阶乘入门例子之外，设计六组 SysY 源程序，并手写对应的 LLVM IR。原始源文件位于 `prep/llvm_ir/`，每组包含同名的`.sy` 和`.ll`。优化 IR 与自动生成的 RISC-V 汇编单独放在 `results/`，避免与手写文件混淆。

实现遵循课程提供的 SysY 定义及文法补充：整数采用 i32，浮点采用 float，普通表达式中的逻辑非结果需转换为整数。LLVM IR 的局部寄存器值遵循 SSA，而保存在内存中的变量还需要 load/store；有这些访存操作并不意味着整个 LLVM IR 是非 SSA。控制流中的合流值通过 phi 表示，具体语义参见 LLVM 语言参考手册 [1]。

2.2 实验环境与复现流程

本次实际环境为 Ubuntu WSL、x86_64，Ubuntu 26.04.1 LTS，Clang/LLVM 21.1.8、RISC-V Linux GCC 15.2.0、QEMU 10.2.1。此环境与搭档已有的 Clang 14 环境分别记录，不能混用版本或编译产物来源。RISC-V 目标使用 RV64GC、LP64D 调用约定与静态链接。

本实验链接已有最小教学运行库 `prep/manual/sylib.c` 中的 getint、putint 和 putch。该实现足以支撑当前例子，但尚未取得课程正式运行库，因此未声称完成正式运行时的全部接口或 ABI 验收。正式库取得后需按同一流程复测。`.sy` 文件不包含 C 头文件；参考编译通过命令行强制包含库声明，且所选示例均避开 SysY 与 C 语义不同的语法。

```
python3 prep/llvm_ir/verify.py
llvm-as sample.ll -o sample.bc
opt -passes=verify -disable-output sample.ll
opt -S '-passes=default<O2>' sample.ll -o sample_O2.ll
llc -mtriple=riscv64-linux-gnu -mattr=+m,+a,+f,+d,+c \
    -target-abi=lp64d sample.ll -o sample_generated_rv.s
```

上面的 sample 命令说明单组操作，实际完整命令由验证脚本记录。脚本每次重新构建程序，检查编译退出码，对运行设置超时，逐项比较标准输出及退出码；不依赖已有可执行文件。二进制写入 build，命令与诊断写入 `commands.json`，输入和预期/实际输出写入 `latest.json`。

示例	主要覆盖	输入数
arithmetic	加减乘除与余数、单目正负、常量、块遮蔽、逻辑非转整数	6
short_circuit	if/else、逻辑与或短路、全局副作用、自定义函数	6
loop_control	while、continue、break、合流与多条回边	6
functions	带参函数、递归、整数返回、void 函数及返回	5
array_float	二维数组、部分初始化、数组形参、浮点运算与转换	5
promote_demo	内存形式循环及 mem2reg 对照	5

表 1: 本次手写 LLVM IR 的实验设计

2.3 示例设计与特性覆盖

算术例子输入 n，计算 $(n+3)\times 2-n/2+n\%3+7$，并分别输出 $-n$、$+n$、$n+!n$。内层同名变量固定为 7，外层 n 在后续输出中仍保持原值，用于验证遮蔽而非覆盖。负数整除按向零截断计算，余数为 $n-\mathrm{trunc}(n/3)\times 3$；不能直接用 Python 向下取整除法作为预期。逻辑非先由 icmp 产生 i1，再由 zext 转为 i32 后参与加法。

2.4 短路控制流与可观测副作用

短路例子包含 `n != 0 && 12 / n > 2` 和 `n == 0 || touch()`。touch 每次调用将全局 hits 加一。以 n=0 为例，正确程序既不除以零，也不调用 touch，最终 hits=0；非零输入则调用 touch 一次。这比只比较纯表达式的真假更能检验执行顺序。

```
%nonzero = icmp ne i32 %n, 0
br i1 %nonzero, label %and.rhs, label %and.false
and.rhs:
  %q = sdiv i32 12, %n
  %large = icmp sgt i32 %q, 2
  br i1 %large, label %and.true, label %and.false
and.join:
  %and.value = phi i32 [1, %and.true], [0, %and.false]
```

代码为关键片段，省略的基本块和声明见同名完整源文件。右操作数置于独立块中，而不是先执行两边再用 and/or 合并；后者会错误执行除法或副作用。测试 n=-3、0、1、4、5、12，覆盖受保护分母、条件真/假和副作用次数。

2.5 循环、函数与数据表示

循环例子逐个处理 1 到 n，跳过 2，在数值超过 5 时退出。continue 的边直接回到循环头，break 的边直接进入出口。循环头分别对 i 和 sum 建立 phi，continue 边保持 sum，普通边传入累加后的值。出口的 phi 对正常条件退出和 break 退出分别提供值。

```
header:
  %i = phi i32 [0, %entry], [%next, %skip], [%next, %accumulate]
  %sum = phi i32 [0, %entry], [%sum, %skip], [%added, %accumulate]
```

```
  %more = icmp slt i32 %i, %n
  br i1 %more, label %body, label %done
done:
  %result = phi i32 [%sum, %header], [%sum, %break.test]
```

LLVM verifier 能发现类型、前驱和支配关系等结构错误，但结构合法不能证明语义正确；还需运行比较。输入 n=2 时只累加 1，n=5 或 n=8 时结果均为 13，分别触发 continue 与 break。函数例子进一步使用递归阶乘和 void 输出函数，检查参数、call 和 ret 的对应关系。

数组/浮点例子将 `int a[2][3]=1,2,3,4` 表示为二维数组对象，初始化第二行剩余元素为零。数组实参传地址，按 `[3 x i32]` 的行类型通过 getelementptr 寻址，两行和为 10。浮点过程为 $\mathrm{twice}(1.5n + 0.5) = 3n + 1$，使用 sitofp、fmul、fadd、fptosi 和 fcmp，既验证 int/float 转换，也验证 float 参数和返回值。本次只选小整数输入，使数值可精确表示；未将浮点舍入误差当作本实验结论。

2.6 mem2reg 观察与结果验证

另手写一份以 alloca/load/store 保存循环变量的求和程序，使用 LLVM 自带 `opt -passes=mem2reg` 得到可提升变量的寄存器形式。转换前为 2 个 alloca、3 个 load、4 个 store、0 个 phi；转换后这些局部访存操作均消失，形成 2 个 phi。对 n=-3、0、1、5、10 分别运行，结果与原程序一致。该实验是对现成 pass 的观察，不代表本小组已自行实现编译器的 mem2reg 模块。

每个输入比较七条路径：参考程序本机/RV64、手写 IR 本机/RV64（原始和 O2），以及链接库后的 lli 运行。33 组输入产生 231 次运行比较，再对 mem2reg 结果做 5 次独立检查，共 **236/236 通过**，均检查 stdout 和退出码。

示例	运行比较数	通过数
arithmetic / short_circuit / loop_control	各 42	各 42
functions / array_float	各 35	各 35
promote_demo（含 mem2reg）	40	40
合计	236	236

表 2: 2026-10-11 实际执行结果

2.7 结论与搭档交接

实验说明手写 IR 的结构正确性、短路顺序、循环合流、数据表示和目标调用约定可以通过分层验证共同检查。优化前后相同输出是必要的正确性证据，但有限测试不能证明任意输入下等价。阶乘限定 0–12，其他输入避免 int 溢出、数组越界、除零和浮点转整数超范围；无效输入处理不在当前验证范围内。

本次 RISC-V 汇编由 llc 自动生成并经 QEMU 检验，用于验证 IR 可进入目标工具链。**这些文件不能替代搭档负责的手写汇编任务。**搭档可按本目录的 SysY 源码和预期输出编写相应

汇编，再把结果接入同一验证矩阵。原先报告中的循环阶乘入门例子仍可保留；扩展例子的完整运行证据已随项目提供。

3 进阶要求：MLIR 多级中间表示探索

3.1 探索目标与证据边界

本部分由袁煜杰完成，研究预备工作进阶要求中的 MLIR 中端处理。首先查阅课件指定的 AscendNPU IR VecAdd 示例与方言文档，再构造上游 MLIR 的同类逐元素加法程序，真实执行多层降低、保存检查点，并验证末端运行结果。两条路线分别是官方 NPU 文档研究与本机 CPU 实验，不将后者写成前者的上板结果。

本机未发现 bishengir-compile、CANN 工具链目录或 NPU 设备节点。课件允许环境配置困难时通过文档阅读、方言差异和探索过程完成进阶说明。因此 Ascend 专有流程采用文档研究，不编造设备输出或性能数据。另从 Ubuntu 官方源下载 MLIR 21.1.8 的工具和共享库，解压到项目临时目录，不安装到系统；由脚本设置进程内的共享库搜索路径。

3.2 AscendNPU IR VecAdd 的方言与层级

官方快速入门 [2] 的 VecAdd 使用 i16 元素，并区分 GM 与 UB 地址空间。输入先搬入片上缓冲，再进行向量加法，最后搬回输出；设备入口用 HACC 属性表达。编译后还需要 CANN runtime 注册和启动设备程序。这与仅把数组指针传给 CPU 函数的执行环境不同。

架构说明 [3] 把 HFusion、HIVM、HACC 分为高层融合、硬件相关计算/搬运/同步、异构执行表达等职责。结合 HIVM 文档 [4]，可以看到片上内存和计算流水信息在特定层被显式保留。**快速入门直接给出 HIVM 级程序，不能由该输入声称已经实际观察到 HFusion 到 HIVM 的转换。**本报告的层级关系来自官方架构说明，非本机 pass 日志。

比较点	上游 CPU 实验	Ascend 官方文档路线
高层计算	Linalg 通用逐元素计算	HFusion 等较高层表达
硬件相关操作	普通 MemRef 访存	HIVM 中的计算、搬运、同步
内存空间	本机普通内存	GM、UB 等硬件空间
执行接口	MLIR 生成 C 包装函数	HACC 入口及 CANN runtime
证据	本机各层输出与运行检查	官方示例和架构文档分析

表 3: 两种路线的职责与证据差异

3.3 可执行的上游 VecAdd 设计

独立设计的 `prep/mlir/vecadd.mlir` 使用三个 `memref<16xi16>` 参数。Linalg 的三个恒等索引映射表示 $c[i] = a[i] + b[i]$，迭代属性 parallel 表明各元素计算互相独立。这保留了后续选择循环或向量化策略的空间，但本次选择的是标量循环路线。

```
linalg.generic {
  indexing_maps = [affine_map<(i)->(i)>,
```

```
                  affine_map<(i)->(i)>, affine_map<(i)->(i)>],
  iterator_types = ["parallel"]
} ins(%a, %b : memref<16xi16>, memref<16xi16>)
  outs(%c : memref<16xi16>) {
^bb0(%x: i16, %y: i16, %old: i16):
  %sum = arith.addi %x, %y : i16
  linalg.yield %sum : i16
}
```

这是本次手写的上游例子，不是复制或执行 Ascend 专有程序。输入数组设为 $a[i] = i$、$b[i] = 1$（$0 \le i < 16$），C 驱动逐项检查输出。

3.4 逐层 lowering 与检查点

上游 MLIR 文档介绍结构化 SCF 与非结构化 CF 的区分 [5]。本次根据工具可用 pass[6] 执行如下管线，每一步均保存文件：

```
mlir-opt vecadd.mlir -o 00_linalg.mlir
mlir-opt 00_linalg.mlir --convert-linalg-to-loops -o 01_scf.mlir
mlir-opt 01_scf.mlir --convert-scf-to-cf -o 02_cf.mlir
mlir-opt 02_cf.mlir --convert-to-llvm \
    --reconcile-unrealized-casts -o 03_llvm.mlir
mlir-translate --mlir-to-llvmir 03_llvm.mlir -o 04_llvm.ll
```

完整脚本 `prep/mlir/run_lowering.py` 支持系统工具或项目内便携工具。上述路径在脚本中落实为 results 下的检查点，不依赖工作目录的绝对用户名。

检查点	行数	实际观察到的操作方言
00_linalg	12	Linalg、Arith、Func
01_scf	15	SCF、MemRef、Arith、Func
02_cf	21	CF、MemRef、Arith、Func
03_llvm	67	LLVM

表 4: 实际保存的 MLIR 检查点（不把接口属性计为操作）

首先 generic 被展开为 `scf.for`，循环体出现两个 memref.load、一次 arith.addi 和一次 memref.store。接着循环被拆成比较、条件分支、循环体和回边，迭代变量通过块参数传递；结构化循环这个概念不再显式保留。再向 LLVM 方言转换时，MemRef 描述符中的已分配地址、对齐地址、偏移、长度和步长被显式展开，生成与 C 调用者衔接的包装函数。最后 `mlir-translate` 才输出真正的 LLVM IR。因此 `llvm.*` 操作所在的 MLIR 文件，与 LLVM 自身的 `.ll` 文件并不是同一种文本格式。

3.5 末端验证与分析

C 驱动构造与一维 MemRef 对应的描述符，通过 `_mlir_ciface_vecadd` 调用降低后的程序。本次 LLVM verifier 检查通过，Clang 编译链接后实际输出：

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
```

驱动逐个断言全部 16 个元素正确，退出码 0。版本、命令、stderr 和结果保存于 `prep/mlir/results/`。行数从 12 增至 67，主要由内存描述符和接口展开造成；该数字不能直接说明运行速度。本次没有测量 NPU 性能，也未执行片上搬运或同步操作。

多级 IR 的价值在于暂时保留当前优化需要的抽象，再逐步选择具体实现。高层可以讨论索引映射和融合，中层明确循环和内存行为，低层处理指针与调用约定。若直接从源程序进入 LLVM IR，这些高层信息通常需要重新推断。与此同时，降低并非仅改指令名称：类型、控制流、内存表示与接口也必须同步转换。上游工具未注册 HIVM/HACC 方言，不能简单套用本次 CPU pass 管线编译官方 NPU 输入。

4 交付物与后续整合

原始手写代码、参考源码、运行脚本、命令记录和分层输出均保存在项目中。LLVM IR 复现入口为 `prep/llvm_ir/verify.py`，MLIR 复现入口为 `prep/mlir/run_lowering.py`；两个目录的 README 说明工具、运行方式与验证范围。共同报告入口为 `LaTeX 模板/main.tex`，本附件由 `scripts/build_yuan_report.py` 从相同章节生成，以保持内容一致。后续由搭档补齐手写汇编对应示例与工作流程分析，双方取得课程正式运行库后再次验证，再统一共同报告并分别提交。当前可作为后续 IR 生成器和目标代码生成器的对照样例。

参考文献

[1] LLVM Project. LLVM Language Reference Manual. https://llvm.org/docs/LangRef.html.

[2] AscendNPU IR. 快速入门示例. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/quick_start/examples_zh.html.

[3] AscendNPU IR. 架构介绍. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/architecture_zh.html.

[4] AscendNPU IR. HIVM Dialect. https://ascendnpu-ir.gitcode.com/zh_cn/sources/developer_guide/dialects/HIVMDialect.html.

[5] MLIR Project. SCF Dialect. https://mlir.llvm.org/docs/Dialects/SCFDialect/.

[6] MLIR Project. Passes. https://mlir.llvm.org/docs/Passes/.